SONATRACH

Innovation center

Ci/Cd Lab Documentation

TABLE OF CONTENT

1. Overview: 3
2. Workflows: 3
   2.1 Infra automation workflow: 3
   2.2 CI/CD workflow: 4
   2.3 Image promotion workflow: 4
3. Components: 5
   3.1 Terraform: 5
   3.2 Kubernetes: 5
   3.3 Argocd/fluxcd: 5
   3.4 Gitlab (setup 1): 5
   3.5 Gitea (setup 2): 6
   3.6 Minio (setup 2): 6
   3.7 Ansible (setup 2): 6
4. Setup1 (terraform, Kubernetes, argocd and gitlab): 6
   4.1 VMs and network flow matrix: 6
   4.2 Deployment guide: 7
5. Setup2 (terraform, ansible, Kubernetes, fluxcd and gitea) 10
   5.1 VMs and network flow matrix: 10
   5.2 Deployment guide: 10
6. Adding a project for the setup to manage 12
7. setups comparison: 13
8. Credentials: 13

LIST OF FIGURES

Figure 1: Infra automation workflow 3
Figure 2: CI/CD workflow 4
Figure 3: Image promotion workflow 4
Figure 4: ArgoCD applications overview (Setup 1) 9
Figure 5: ArgoCD, state of all pods of an application (Setup 1) 9

LIST OF TABLES

Table 1: VMs and network flow matrix (Setup 1) 6
Table 2: VMs and network flow matrix (Setup 2) 10
Table 3: Credentials 13

1. Overview:

This document is about the two setups that aim to achieve a complete CI/CD and GitOps setup that runs entirely on premise, and promises auto scaling and healing worker vms of a Kubernetes cluster, where apps in the repo are automatically deployed into said cluster.

The first setup uses (terraform, gitlab, argocd and kubernetees) and the other (setup2) uses (terraform, kubernetes , fluxcd, gitea and ansible), so this document will go through the general ideas/workflows and then move on to detailing these two variations.

2. Workflows:

This document has three main work flows about the infra-automation, CI/CD and promoting an image from a namespace to another (simulating the promotion process from staging to production)

We will go into more details in the actual setup.

2.1 Infra automation workflow:

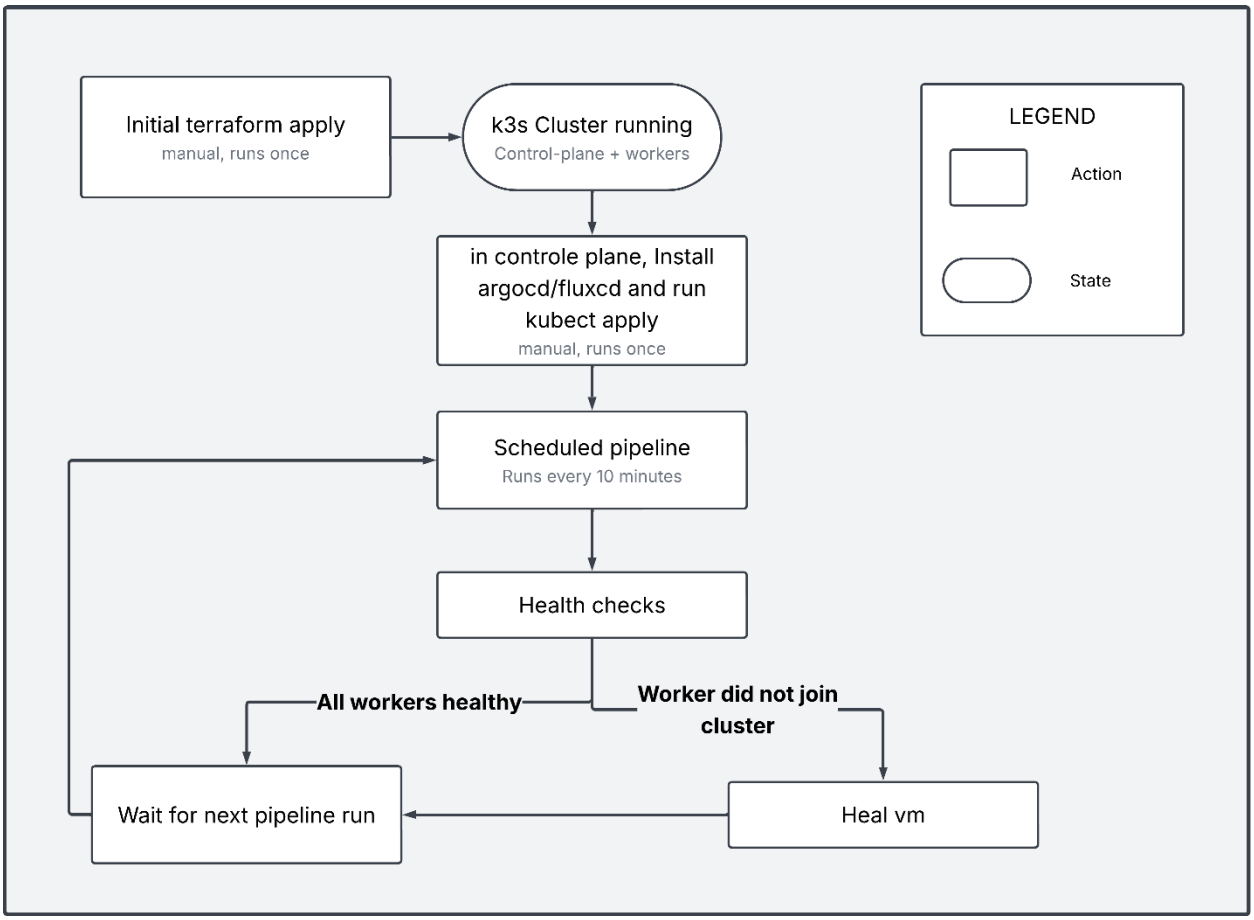


Figure 1: Infra automation workflow

2.2 CI/CD workflow:

The ci/cd has two main models (push model and pull model),
The push model is where once there is a change in the repo, the ci/cd pipeline applies said changes to the cluster, and the pull model (what we used) is where the cluster (through fluxcd/argocd) monitors the repo and once it detected changes it marks the current build as asynchronous, pulls the changes from the repo and tries to synchronize it.

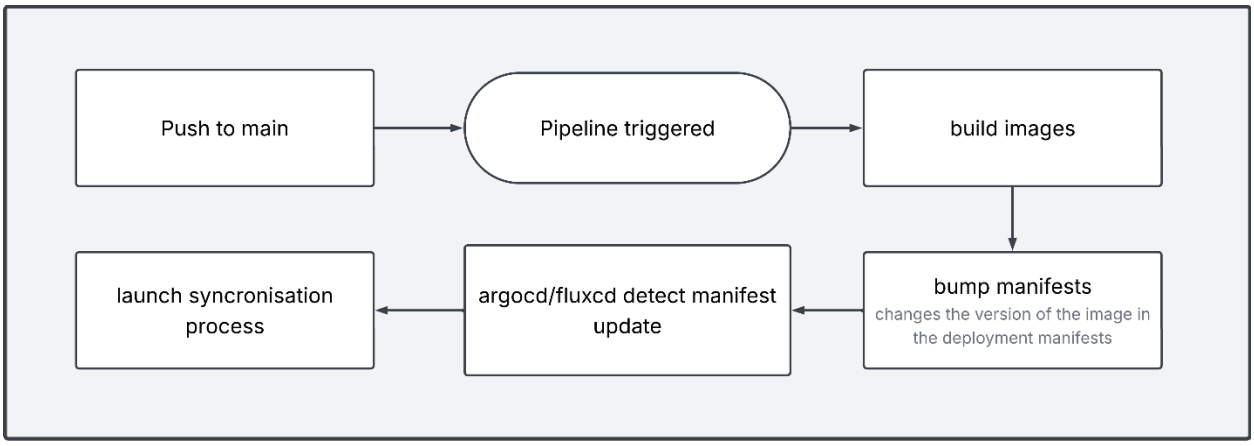


Figure 2: CI/CD workflow

Note: currently the pipeline trigger is any push to the main branch, this should be changed to what suits the dev team.

2.3 Image promotion workflow:

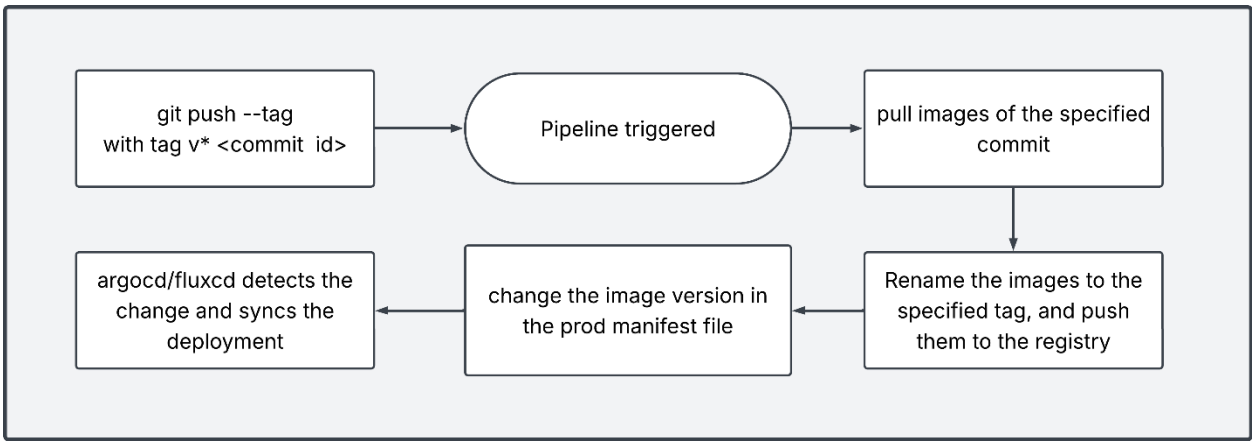


Figure 3: Image promotion workflow

3. Components:

3.1 Terraform:

Terraform is the Infra-as-Code layer for the GitLab setup.

Roles:

- From a prepared template (a vm created form image jammy-server-cloudimg-amd64.img), we use terraform to create the vms of the k3s cluster, and we configure said vms using cloud init file.
- Is invoked in two distinct contexts: a one-time manual initial apply to bootstrap the cluster, and repeated applies triggered by the scheduled monitoring pipeline (worker healing, scale up/down)

Notes:

- whenever u (terraform apply) you create a state file, that file is needed for any further terraform commands on the created vms, so it must be available.
  that availability is provided by gitlab (setup1) and minio (setup2)

3.2 Kubernetes:

We used k3s (a lighter distribution), is the container orchestration layer

3.3 Argocd/fluxcd:

Argocd is manually installed inside the Kubernetes cluster.

Roles:

- It is what deploys/manages the apps that we want to deploy inside the cluster.
- It monitors the manifests repo of the deployed apps, and once it detects changes it reconciles with the deployed version and synchronise it.

Notes:

- Adding an app is a manual process, of cloning the manifest repo and applying (kubectl apply -f path) the aregocd/fluxcd directory.

3.4 Gitlab (setup 1):

Roles:

- Is a git hosting platform, that contains our repos
- Provides a container registry where our built images are saved
- Provides Terraform state backend, that makes the state file accessible
- GitLab pipelines are used for infrastructure monitoring and self-healing, as well as CI/CD.

Notes:

- The ci/cd pipeline need to be added for each app (not a single general one)
- Gitlab requires at least one runner (a different software ran as a container and registered into gitlab), and said runner is what execute the pipelines and other jobs.

3.5 Gitea (setup 2):

Roles:

- Is a git hosting platform, that contains our repos
- Provides a container registry where our built images are saved
- GitLab pipelines are used for infrastructure monitoring and self-healing, as well as CI/CD.

Notes:

- The ci/cd pipeline need to be added for each app (not a single general one)

3.6 Minio (setup 2):

Minio is an s3 solution, used here solely as the state backend for terraform

3.7 Ansible (setup 2):

Currently in setup 1, if a worker failed to join the cluster the whole worker vm is destroyed, and recreated, which is awfully inefficient.
So, in setup2 we used ansible in the infra pipeline to manage the service (k3s agent) and runs the tests (internet access, disk space, k3s agent state …ect), so if the worker failed to join the cluster ansible will first try to fix the k3s agent, and if it fails, we simply destroy and recreate the vm, which a way more efficient approach.

4. Setup1 (terraform, Kubernetes, argocd and gitlab):

4.1 VMs and network flow matrix:

Table 1: VMs and network flow matrix (Setup 1)

VM	Hostname	IP Address	Containers
gitlab	Admin	192.168.10.173	Gitlab, gitlab runner
k3s-controle-plane	Ubuntu	192.168.10.177	/
k3s-worker-*	Ubuntu	dhcp	/

The flow matrix is available as a separate excel file.

4.2 Deployment guide:

4.2.1 Step1: preparations

Create the VM cicd1-gitlab (any linux server os)
(in the rest of the guide, ./ points to ./cicd setup1).

4.2.2 Step2: GitLab

- Move the ./gitlab-setup/ folder to the cicd1-gitlab VM
- Execute the commands

```
sudo docker compose up
sudo  docker exec -it gitlab grep 'Password:' /etc/gitlab/initial_root_password
```

- Log in as (root: "password gotten from the second commnad"), then change the password
- Create a Group, then create three Projects inside it: app, infra, manifests
- Push our three repos into the created repos

4.2.3 Step3: Register runner

- Go to the Group → Settings → CI/CD → Runners
- Create new runner, and copy the glrt-... token.
- Run:

```
docker exec -it "<GitLab Runner container name>" gitlab-runner register \
      --url "<Gitlab url>" \
      --token "<RUNNER_TOKEN>" \
      --executor "docker" \
      --docker-image "docker:24" \
      --docker-volumes "/var/run/docker.sock:/var/run/docker.sock"
```

- Check the runner shows as Idle in the page where u created the token

4.2.4 Step4: create initial k3s cluster using terraform:

- Check the proxmox url, token id and secret, ..ect in the file ./repos/infra/terraform.tfvars and also modify the used public key if u don't have its private key.
- In the files of the directory ./repos/infra/cloud-inti/, check the used urls (it specifies the insecure registries)
- Create an access token for terraform (GitLab top-right avatar→ Edit profile→ Access Tokens) With the scope "check api"
- The next need to be executed in a machine with terraform installed and with access to the proxmox url

- Run:

```
cd ./infra/
export TF_HTTP_ADDRESS="http://<gitlab ip>/api/v4/projects/2/terraform/state/default"
export TF_HTTP_LOCK_ADDRESS="$TF_HTTP_ADDRESS/lock"
export TF_HTTP_UNLOCK_ADDRESS="$TF_HTTP_ADDRESS/lock"
export TF_HTTP_USERNAME="root"
export TF_HTTP_PASSWORD="<TOKEN HERE>"
export TF_HTTP_LOCK_METHOD="POST"
export TF_HTTP_UNLOCK_METHOD="DELETE"

terraform init
terraform apply
```

note: if u want to execute a script right after creating the vms (eg: internet config), add it to the cloud init file (./repos/infra/cloud-init/).

- Now you can access the created vms using the private ssh key of the public key specified in the terraform.tfvars
- Check the cluster state (ssh into the k3s-controle vm, and run kubectl get nodes)
  if you see all the workers as ready then it is ready.

4.2.5 Step5: ArgoCD

- SSH into the k3s-controle vm
- Run:

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-
cd/stable/manifests/install.yaml
```

- Then check the state of argocd using “kubectl get pods -n argocd”
- Next we need to clone the repo (manifest)

```
git clone http://<gilab ip>/<organisation>/manifests.git
cd manifests/
kubectl apply -f ./argocd/
```

- The last kubectl apply creates the secrets and the ‘App’ object in argocd namespace
- To check the state of the deployed apps argo cd provides a web interface
  to get its port run: “kubectl get svc -n argocd” and look for the port of the argocd server pod

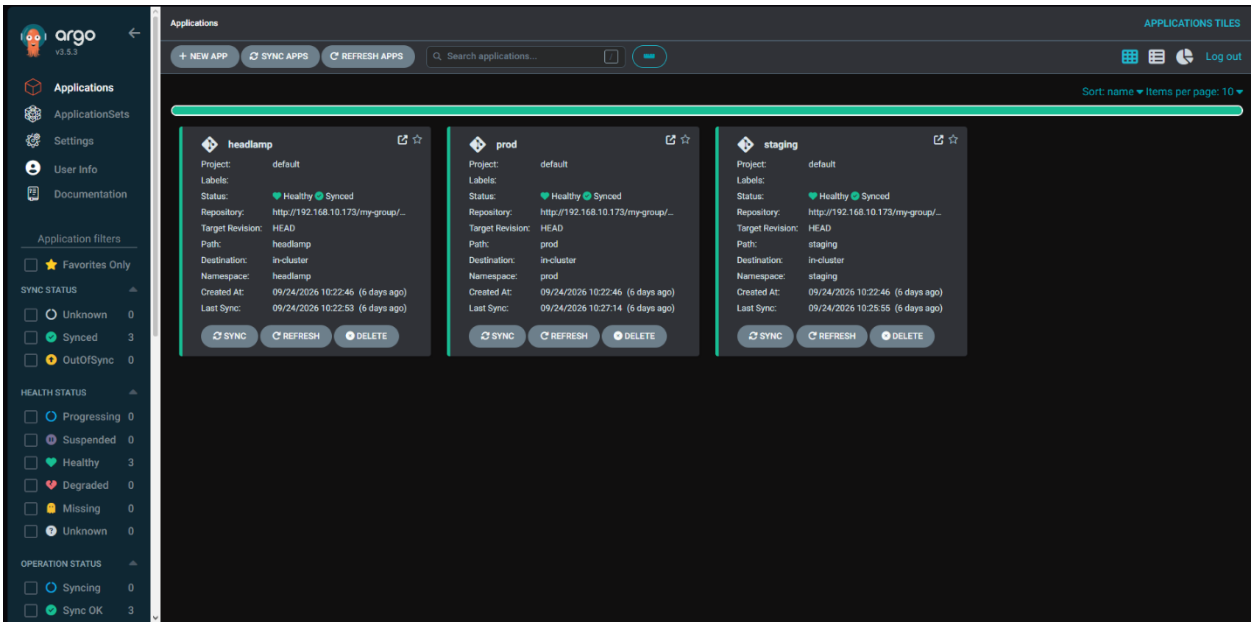


Figure 4: ArgoCD applications overview (Setup 1)

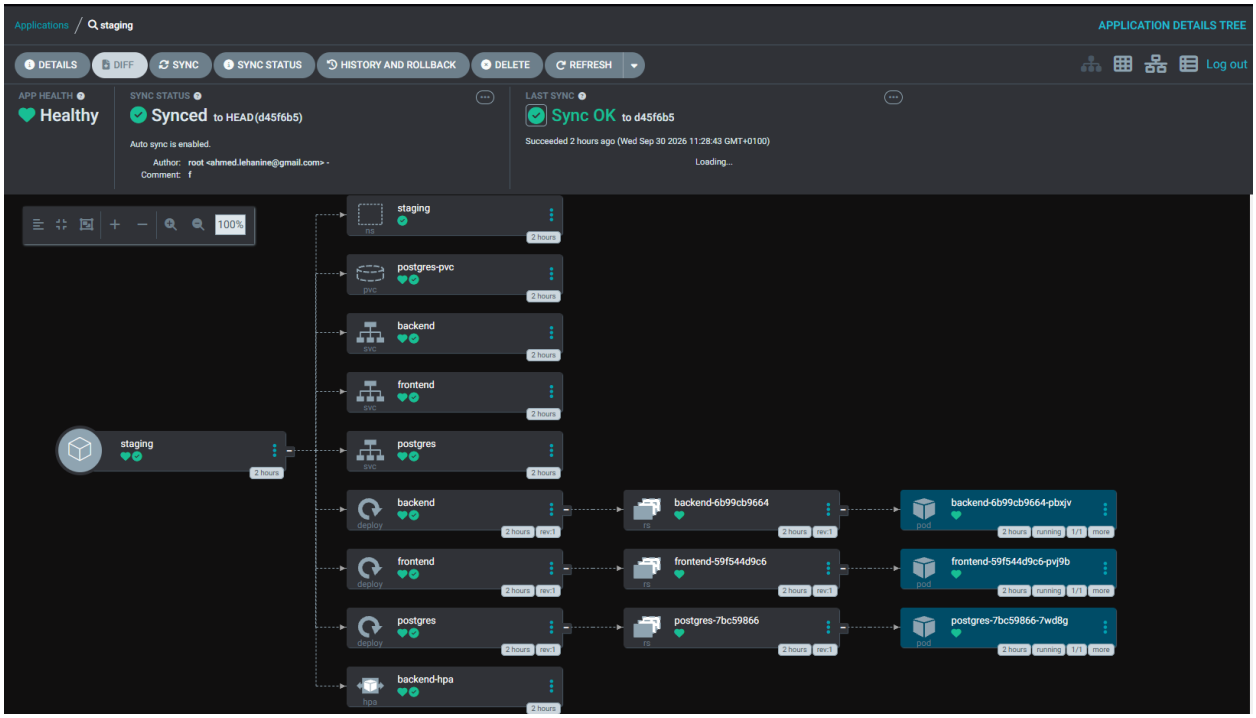


Figure 5: ArgoCD state of all pods of an application (Setup 1)

4.2.6 Step6: infra monitoring/healing pipeline

- Add the following variables in the page (open the repo infra→ Settings→ CI/CD → Variables): SSH_PRIVATE_KEY (of the k3s cluster vms), PROXMOX_API_TOKEN, PVE_TOKEN_SECRET, PROXMOX_URL
- Add a pipeline scheduler.

5. Setup2 (terraform, ansible, Kubernetes, fluxcd and gitea)

5.1 VMs and network flow matrix:

Table 2: VMs and network flow matrix (Setup 2)

VM	Hostname	IP Address	Containers
cicd2-gitea	Ubuntu	192.168.10.173	Gitlab, gitlab runner
cicd2-K3s-controle-plane	Ubuntu	192.168.10.177	/
cicd2-K3s-worker-*	Ubuntu	dhcp	/
cicd2-minio	Ubuntu	192.168.10.89	minio

The flow matrix is available as a separate excel file.

5.2 Deployment guide:

5.2.1 Step1: preparations

Create the vms cicd2-minio and cicd2-gitea, (in the rest of the guide ./ points to ‘./cicd setup2’)

5.2.2 Step2: minio

- Move the ./minio/ folder to the cicd2-minio vm
- Execute the commands

```
cd ./minio/
sudo docker compose up
```

- Open http://<MinIO-VM-IP>:9001, and login with the admin user specified in the docker-compose.yml file.
- Create a bucket named “terraform-state”
- Create access key to the previously created bucker
- Paste the key credentials into the ./repos/infra/main.tf (access-key and secret-key fields of the backend object)

5.2.3 Step3: gitea

- Move the ./gitea-setup/ folder to the cicd2-gitea vm
- Execute the commands

```
cd ./gitea-setup/gitea/
sudo docker compose up
```

- Browse to http://<minio vm ip>:3000, create an organization, and push our 3 repos into the

organization.

- Go to organization – settings – actions – runners – create new runner, then paste the token into the docker compose of the runner, and docker compose up it

```
cd ./gitea-setup/gitea-runner/
sudo docker compose up
```

- check that the runner is considered idle in the page where you created it
- Push our three repos into the organization.

5.2.4 Step4: create initial k3s cluster using terraform:

- Check the proxmox url, token id and secret, ..ect in the file ./repos/infra/terraform.tfvars and also modify the used public key if u don't have its private key.
- In the files of the directory ./repos/infra/cloud-inti/, check the used urls (it specifies the insecure registries)
- The next need to be executed in a machine with terraform installed and with access to the proxmox url
- run terraform apply (after cd into infra directory),

```
cd ./infra/
terraform apply
```

note: if u want to execute a script right after creating the vms (eg: internet config), add it to the cloud init file (./repos/infra/cloud-init/).

- Now you can access the created vms using the private ssh key of the public key specified in the terraform.tfvars
- Check the cluster state (ssh into the k3s-controle vm, and run kubectl get nodes)
  if you see all the workers as ready then it is ready.

5.2.5 Step5: FluxCD

- SSH into the k3s-controle vm
- Run:

```
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
curl -s https://fluxcd.io/install.sh | sudo bash
flux check –pre
flux install
```

- Then check the state of fluxcd using "kubectl get pods -n flux-system"

Note: to have access to the command 'flux' u need the to "export KUBECONFIG=.." in that ssh session.

- Next we need to clone the repo (manifest)

```
git clone http://<gitea ip>:3000/<organisation>/manifests.git
cd manifests/
kubectl apply -f ./flux/
```

- The last kubectl apply creates the secrets and the 'Kustomization' object (equivelant to the custom object 'app' used by argo cd) in the specified namespaces
  -note that the kustomization object is what tells flux cd which repo/file-path to monitor and auto deploy
- To check the state of the deployed apps there is no ui, u need to run "kubectl get pods -n <app

namespace>" and in the case where a pod is having an issue u need to run "kubectl describe <pod name> -n <app namespace>", to see the events.

5.2.6 Step6: infra monitoring/healing pipeline

- Add the following variables in the page (open the repo infra→ Settings→ Actions → Secrets → Add Secret):
  SSH_PRIVATE_KEY (of the k3s cluster vms), K3S_TOKEN, MINIO_ACCESS_KEY, MINIO_SECRET_KEY, PROXMOX_API_TOKEN

6. Adding a project for the setup to manage

Adding a new application into either setup requires three things: create a new repo in Gitea/GitLab and upload your project, create the CI/CD pipeline for your project, and write the Kubernetes manifests that describe how it runs in the cluster

6.1 ci/cd pipeline

The CI/CD pipeline is defined by a configuration file in the app's repository. said file depends on the setup:

- **GitLab setup:** create/edit .gitlab-ci.yml
- **Gitea setup:** create/edit .gitea/workflows/<name>.yml (any yml file inside the workflows directory defines a pipeline)

The main functionality of the pipeline is building the project's images (eg: for a 3tier website it would build separate images for the backend and front end), and modifying the manifests to point to the newly built images.
So for the new project's pipeline, you can take the frame of these jobs from the provided web-app and modify them to your use case.

The pipeline can provide other functionalities (eg: premade tests that can be imported, custom test to test your app before pushing the new image …ect), so you can add what suits your own project.

Note: currently the pipeline is triggered by every push to the main branch, that can and should be changed to match your workflow.

6.2 manifests

The manifests repo is what ArgoCD/Flux actually watches, whatever Kubernetes YAML lives in the path your **Application/Kustomization object** points to is what gets deployed to the cluster automatically, whenever it changes. This is the single source of truth for what's actually running.

So for your project you need to create an Application/Kustomization manifest file in manifests/arogocd or manifests/fluxcd, and the manifests of your deployment to Kubernetes in a folder your earlier manifest point towards. Check the existing staging/ and prod/ manifests to see how they are deployed.

Note: after creating the manifest you should go to k3s controle plane and pull the manifest repo and run "Kubernetes apply -f ./manifests/fluxcd" or "Kubernetes apply -f ./manifests/argocd"

7. setups comparison:

The main difference between them is how the monitoring and healing is handeled, and the git host platform:

7.1 monitoring and healing:

In setup one (gitlab), this was handled using bash scripts inside the pipeline while in setup two (gitea) we used a combination of scripts and ansible to handle it, so the second one is way superior in that aspect.

7.2 Git host platform:

GitLab offers all what gitea offers, plus an integrated terraform state backend, better control over the container registry and is more convenient (eg: can directly import packages In the pipeline instead of manually installing them).
But Gitlab is way heavier than gitea (mainly ram utilization) so in the case where ram is limited, gitea can get the job done.

8. Credentials:

Table 3: Credentials

	Service	Username	Password
Setup1	Gitlab Vm	admin	admin
	Vms: k3s-control-plane k3s-worker-x	ubuntu	Ssh private key (remove the extra line breaks): "ssh-ed25519 AAAAC3NzaC1lZDI1 NTE5AAAAILsPjvJykxoVJowMU4W zs3TbUrdOYTERCa4JYagifUql ahmed@Sefirot"
	GitLab admin	root	FlF04FlF
	ArgoCD admin	admin	FlF04FlF
Setup2	Gitea admin	root	root1234
	MinIO root	admin	admin1234
	Vms: cicd2-k3s-control-plane cicd2-k3s-worker-x	ubuntu	Ssh private key (remove the extra line breaks): "ssh-ed25519 AAAAC3NzaC1lZDI1 NTE5AAAAILsPjvJykxoVJowMU4W zs3TbUrdOYTERCa4JYagifUql ahmed@Sefirot"
other	Physical proxmox server	root	root1234